Islamic University of Technology

Organization of Islamic Cooperation



Name	**:** Antara Arifa Mullick
ID	**:** 210042132
Program	**:** SWE
Department	**:** CSE
Subject	**:** Artificial Intelligence
Subject Code	**:** CSE 4618
Title	**:** Multiagent Search
Date of Submission	**:** 18th August, 2025

1. Introduction

This lab is focused on implementing adversarial search algorithms in the context of a multi-agent environment using the classic Pacman game with ghosts as adversaries. The goal was to design intelligent agents capable of making optimal decisions, considering the actions of multiple adversarial agents (ghosts). The project involved the development of four key search algorithms:

- Reflex Agent: A simple agent that makes decisions based on an evaluation function.
- Minimax Search: A decision-making algorithm that optimizes for the worst-case scenario, assuming ghosts act optimally.
- Alpha-Beta Pruning: An optimization to the Minimax algorithm that reduces the search space while maintaining optimality.
- Expectimax Search: A probabilistic version of Minimax, where ghosts are modeled as random agents.

The main challenge of the lab was to develop agents that can balance offensive strategies (e.g., food collection) with defensive strategies (e.g., avoiding ghosts) by reasoning about future game states in a multi-agent environment.

2. Problem Analysis

Multi-Agent Environment Complexity

The Pacman domain presents several unique challenges:

- Multiple Agents: Pacman (the maximizer) interacts with multiple ghosts (the minimizers).
- Turn-Based Sequential Decision Making: The game alternates between Pacman’s move and each ghost's move.
- Variable Depth Search: Each game step (ply) consists of one Pacman move followed by all ghost moves.
- Real-Time Constraints: The agents must make decisions quickly to ensure real-time playability.

Algorithm-Specific Considerations

- Minimax: Assumes optimal play by all ghosts. While it guarantees the worst-case performance bounds, it leads to overly cautious behavior.

- Alpha-Beta Pruning: Maintains the optimality of Minimax but reduces the search space by pruning branches that cannot affect the final decision.

- Expectimax: Models ghosts as random agents rather than optimal adversaries, offering a more realistic approach to playing against suboptimal opponents.

3. Solution Explanation

Question 1: Reflex Agent Enhancement

The Reflex Agent selects actions based on an evaluation function that assigns scores to each possible action. The evaluation function accounts for several factors such as the distance to the nearest food, proximity to ghosts, and the current score.

```
def evaluationFunction(self, currentGameState, action):

    successorGameState = currentGameState.generatePacmanSuccessor(action)

    newPos = successorGameState.getPacmanPosition()

    newFood = successorGameState.getFood()

    newGhostStates = successorGameState.getGhostStates()

    newScaredTimes = [ghostState.scaredTimer for ghostState in newGhostStates]



    foodList = newFood.asList()

        minFoodDistance = min([manhattanDistance(newPos, food) for food in foodList]) if foodList else 1

    ghostPositions = [ghostState.getPosition() for ghostState in newGhostStates]

    ghostDistances = [manhattanDistance(newPos, ghostPos) for ghostPos in ghostPositions]



    activeGhosts = [ghost for ghost in newGhostStates if ghost.scaredTimer == 0]
```

```
    activeGhostDistances = [manhattanDistance(newPos, ghost.getPosition()) for ghost in activeGhosts]

  ghostPenalty = -10 / (min(activeGhostDistances) + 1) if activeGhostDistances else 0

  score = successorGameState.getScore()

  score += 10 / (minFoodDistance + 1)

  score += ghostPenalty

   return score
```

Key design decisions:

- Used reciprocal distances (1/distance) to create stronger gradients near important objects.
- Differentiated between scared and active ghosts for appropriate behavior.
- Balanced food attraction and ghost avoidance through weighted scoring.

Question 2: Minimax Implementation

The Minimax algorithm computes the optimal move by recursively evaluating the game tree. Pacman aims to maximize the score, while the ghosts minimize it.

```
def minimax(state, depth, agentIndex):

  if state.isWin() or state.isLose() or depth == self.depth:

     return self.evaluationFunction(state)

  numAgents = state.getNumAgents()

  nextAgentIndex = (agentIndex + 1) % numAgents

  nextDepth = depth + 1 if nextAgentIndex == 0 else depth

  legalActions = state.getLegalActions(agentIndex)

  if not legalActions:

     return self.evaluationFunction(state)
```

```
    if agentIndex == 0:
        return max(minimax(state.generateSuccessor(agentIndex, action), nextDepth, nextAgentIndex) for action in legalActions)
    else:
        return min(minimax(state.generateSuccessor(agentIndex, action), nextDepth, nextAgentIndex) for action in legalActions)
```

Question 3: Alpha-Beta Pruning

Alpha-Beta Pruning optimizes the Minimax algorithm by pruning branches of the search tree that cannot influence the final decision, improving computational efficiency without sacrificing optimality.

```
def alphaBeta(state, depth, agentIndex, alpha, beta):
    if state.isWin() or state.isLose() or depth == self.depth:
        return self.evaluationFunction(state)
    numAgents = state.getNumAgents()
    nextAgentIndex = (agentIndex + 1) % numAgents
    nextDepth = depth + 1 if nextAgentIndex == 0 else depth
    legalActions = state.getLegalActions(agentIndex)
    if not legalActions:
        return self.evaluationFunction(state)
    if agentIndex == 0:
        value = float('-inf')
        for action in legalActions:
            successor = state.generateSuccessor(agentIndex, action)
            value = max(value, alphaBeta(successor, nextDepth, nextAgentIndex, alpha, beta))
```

```
            if value > beta:
                return value
            alpha = max(alpha, value)
        return value
    else:
        value = float('inf')
        for action in legalActions:
            successor = state.generateSuccessor(agentIndex, action)
            value = min(value, alphaBeta(successor, nextDepth, nextAgentIndex, alpha, beta))
            if value < alpha:
                return value
            beta = min(beta, value)
        return value
```

Question 4: Expectimax Implementation

The Expectimax algorithm treats ghosts as random agents and calculates the expected value of all possible outcomes.

```
def expectimax(state, depth, agentIndex):
    if state.isWin() or state.isLose() or depth == self.depth:
        return self.evaluationFunction(state)

    numAgents = state.getNumAgents()
    nextAgentIndex = (agentIndex + 1) % numAgents
    nextDepth = depth + 1 if nextAgentIndex == 0 else depth
    legalActions = state.getLegalActions(agentIndex)
```

```
    if not legalActions:
        return self.evaluationFunction(state)
    if agentIndex == 0:  # Pacman (maximizing)
        return max(expectimax(state.generateSuccessor(agentIndex, action), nextDepth, nextAgentIndex) for action in legalActions)
    else:  # Ghosts (chance node)
        values = [expectimax(state.generateSuccessor(agentIndex, action), nextDepth, nextAgentIndex) for action in legalActions]
        return sum(values) / len(values)
```

Question 5: Advanced Evaluation Function

The advanced evaluation function incorporates multiple factors, including food proximity, ghost distance, remaining food pellets, and power pellets.

```
def betterEvaluationFunction(currentGameState):
    pos = currentGameState.getPacmanPosition()
    food = currentGameState.getFood()
    ghostStates = currentGameState.getGhostStates()
    scaredTimes = [ghostState.scaredTimer for ghostState in ghostStates]
    capsules = currentGameState.getCapsules()
    score = currentGameState.getScore()
    foodList = food.asList()
    if foodList:
        minFoodDistance = min([manhattanDistance(pos, food) for food in foodList])
        score += 10.0 / (minFoodDistance + 1)
        score -= 4 * len(foodList)
    if capsules:
```

```
            minCapsuleDistance = min([manhattanDistance(pos, capsule) for capsule in capsules])
        score += 50.0 / (minCapsuleDistance + 1)
        score -= 10 * len(capsules)
    for i, ghostState in enumerate(ghostStates):
        ghostPos = ghostState.getPosition()
        ghostDistance = manhattanDistance(pos, ghostPos)
        if scaredTimes[i] > 0:
            score += 200.0 / (ghostDistance + 1)
            score += 2 * scaredTimes[i]
        else:
            if ghostDistance <= 1:
                score -= 500
            elif ghostDistance <= 3:
                score -= 50.0 / (ghostDistance + 1)

    if currentGameState.isWin():
        score += 1000
    if currentGameState.isLose():
        score -= 1000
    return score
```

4. Findings and Insights

Behavioral Insights

- The Minimax agent exhibited overly cautious behavior, often opting for defensive moves, even when aggressive play could have been beneficial.

- The Expectimax agent, on the other hand, exhibited more aggressive behavior, leading to a higher win rate, particularly in scenarios where the Minimax agent would have lost.

Algorithmic Insights

- Minimax vs Expectimax: The decision between Minimax and Expectimax greatly influences the agent's performance. Expectimax, by modeling ghosts as random agents, proved to be more successful in certain scenarios.

- Evaluation Function: The evaluation function played a critical role in improving performance. A well-balanced function with multiple factors, such as food proximity and ghost avoidance, led to better results than relying on search depth alone.

5. Challenges Faced

Technical Implementation Challenges

- Agent Index Management: Cycling through multiple agents and correctly managing the depth at each level of recursion posed challenges, especially when handling Pacman and multiple ghosts.

- Alpha-Beta Pruning: Ensuring the correctness of the pruning conditions while maintaining compatibility with the autograder's expected behavior was a meticulous task.

Performance Optimization

- State Generation Minimization: The autograder closely monitors calls to the `generateSuccessor()` function, requiring efficient handling to avoid redundant state generation.

6. Hyperparameter Exploration

Evaluation Function Weights

I tested different weights for components of the evaluation function:

- Food Distance: Increased from 5 to 10 for better food-seeking behavior.
- Ghost Penalty: Increased significantly to improve ghost avoidance, with extreme penalties for immediate danger.
- Capsule Bonus: Adjusted to prioritize power pellet consumption.

Depth Analysis

- Depth 1: Fast but myopic.
- Depth 2: A good balance between performance and computation time.
- Depth 3: Offered marginal improvement at a significant computational cost.

7. Conclusion

This lab reinforced my understanding of adversarial search in multi-agent environments. By implementing and optimizing Minimax, Alpha-Beta Pruning, and Expectimax, I was able to develop agents that balance offensive and defensive strategies effectively. The experience also highlighted the importance of a well-designed evaluation function in optimizing performance in adversarial settings.